セルフオーダーシステム 制作① メニュー表示

DB の商品を ASP.NET Core MVC の画面へ表示する

Visual Studio 2022 / .NET 8　Version 1.0

この資料は制作手順と実行確認を一つずつ進めるためのレジュメです

到達目標

- SelfOrderSystem プロジェクトを Visual Studio 2022 で作成できる
- Category と Product を EF Core で LocalDB へ保存できる
- SeedData で初期商品を登録できる
- MenuController から ViewModel を View へ渡して商品一覧を表示できる
- カテゴリを選択して表示商品を絞り込める

【前提】ASP.NET Core・MVC 入門、Razor 入門、EF Core・LocalDB 入門の内容を利用する

【重要】ここからは練習用の MvcBasicSample ではなく、SelfOrderSystem という新しいプロジェクトを作成する。

STEP 1　プロジェクトを作成する

対象ファイル	SelfOrderSystem
作業区分	新規作成
目的	ASP.NET Core MVC の制作プロジェクトを準備する

操作手順

1. Visual Studio 2022 で［新しいプロジェクトの作成］を選択する
2. ［ASP.NET Core Web App (Model-View-Controller)］を選択する
3. プロジェクト名を SelfOrderSystem とする
4. フレームワークは .NET 8.0、認証は［なし］を選択して作成する
5. F5 でテンプレートの Home 画面が表示されることを確認して停止する

実行確認

- テンプレートの Home 画面が表示される
- ソリューション エクスプローラーに Controllers / Models / Views / wwwroot がある

STEP 2　EF Core のパッケージを追加する

対象ファイル	SelfOrderSystem.csproj
作業区分	既存プロジェクトへ追加
目的	SQL Server LocalDB を EF Core から扱えるようにする

操作手順

6. プロジェクトを右クリックして［NuGet パッケージの管理］を開く
7. Microsoft.EntityFrameworkCore.SqlServer の 8 系を追加する
8. Microsoft.EntityFrameworkCore.Tools の 8 系を追加する
9. Microsoft.EntityFrameworkCore.Design の 8 系を追加する
10. EF Core 関連パッケージのメジャーバージョンが 8 でそろっていることを確認する

> **【注意】**EF Core 9 や 10 を混在させない。.NET 8 では EF Core 8 系に統一する。

STEP 3　フォルダーを準備する

対象ファイル	Data / Services / ViewModels
作業区分	新規フォルダー
目的	役割ごとにファイルを整理する

操作手順

11. プロジェクト直下に Data、Services、ViewModels の 3 フォルダーを作成する

コードの意味

Data：EF Core の DbContext や初期データを置く

Services：画面から分離した処理を置く

ViewModels：View へ渡す画面専用データをまとめる

STEP 4　商品データの Model を作る

対象ファイル	Models/Category.cs / Models/Product.cs
作業区分	新規作成
目的	カテゴリと商品を C#のクラスで表現する

完成コード

```
using System.ComponentModel.DataAnnotations;
```

```
namespace SelfOrderSystem.Models;

public class Category
{
    public int Id { get; set; }                                    // 主キー

    [Required]
    [StringLength(50)]
    public string Name { get; set; } = string.Empty;               // カテゴリ名

    public int SortOrder { get; set; }                             // 表示順

    public List<Product> Products { get; set; } = new();           // 所属商品
}
```

完成コード

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace SelfOrderSystem.Models;

public class Product
{
    public int Id { get; set; }                                    // 主キー
    public int CategoryId { get; set; }                            // 外部キー

    [Required]
    [StringLength(100)]
    public string Name { get; set; } = string.Empty;               // 商品名

    [Column(TypeName = "decimal(10,0)")]
    public decimal Price { get; set; }                             // 価格

    [StringLength(500)]
    public string? Description { get; set; }                       // 商品説明

    [StringLength(300)]
    public string? ImagePath { get; set; }                         // 商品画像パス

    public bool IsSoldOut { get; set; }                            // 売り切れ
    public bool IsActive { get; set; } = true;                     // 表示対象

    public Category? Category { get; set; }                        // 所属カテゴリ
}
```

コードの意味

外部キー：Product.CategoryId が Categories テーブルの Id を参照する

ナビゲーションプロパティ：Product.Category や Category.Products で関連データを C#からたどれる

DataAnnotations：Required や StringLength でデータのルールを表す

STEP 5　AppDbContext を作る

対象ファイル	Data/AppDbContext.cs
作業区分	新規作成
目的	Model と SQL Server を EF Core でつなぐ

完成コード

```
using Microsoft.EntityFrameworkCore;
using SelfOrderSystem.Models;


namespace SelfOrderSystem.Data;


public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }


    public DbSet<Category> Categories => Set<Category>();        // カテゴリ
    public DbSet<Product> Products => Set<Product>();            // 商品
}
```

コードの意味

DbContext：EF Core による DB 操作全体を管理する

DbSet<T>：各テーブルを C#から操作する入口になる

STEP 6　LocalDB の接続文字列を設定する

対象ファイル	appsettings.Development.json
作業区分	既存編集または新規作成
目的	開発 PC の LocalDB へ接続する

完成コード

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=SelfOrderSystemDb;Trusted_Connection=True;MultipleActiveResult
Sets=true;TrustServerCertificate=True"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

【補足】(localdb)¥MSSQLLocalDB は SQL Server LocalDB のインスタンス名、SelfOrderSystemDb は今回作成する DB 名である。

STEP 7　Program.cs へ DB 設定を登録する

対象ファイル	Program.cs
作業区分	既存編集
目的	AppDbContext を ASP.NET Core の DI コンテナへ登録する

完成コード

```
using Microsoft.EntityFrameworkCore;
using SelfOrderSystem.Data;


var builder = WebApplication.CreateBuilder(args);


builder.Services.AddControllersWithViews();


builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));


var app = builder.Build();


if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}


app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();


app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Menu}/{action=Index}/{id?}");            // 既定画面を Menu にする


app.Run();
```

【重要】この時点では MenuController がまだないため F5 で 404 になることがある。ビルドエラーとは別なので、まず［ビルド］→［ソリューションのビルド］でエラー0 件を確認する。

STEP 8　Migration で DB を作成する

対象ファイル	Package Manager Console
作業区分	コマンド実行
目的	Model から SelfOrderSystemDb を作る

操作手順

12. ［ツール］→［NuGet パッケージ マネージャー］→［パッケージ マネージャー コンソール］を開く
13. Add-Migration InitialCreate を実行する
14. Build succeeded. を確認する
15. Update-Database を実行する
16. SQL Server オブジェクト エクスプローラーで SelfOrderSystemDb を確認する

【重要】Add-Migration は変更内容を Migration として記録し、Update-Database がその内容を実際の DB へ反映する。

STEP 9　初期データを登録する

対象ファイル	Data/SeedData.cs / Program.cs
作業区分	SeedData 新規作成・Program.cs 編集
目的	DB を空のままにせず動作確認用の商品を登録する

完成コード

```
using Microsoft.EntityFrameworkCore;
using SelfOrderSystem.Models;


namespace SelfOrderSystem.Data;


public static class SeedData
{
    public static async Task InitializeAsync(AppDbContext db)
    {
        if (await db.Categories.AnyAsync())
        {
            return;                                                 // 登録済みなら終了
        }


        var meat = new Category { Name = "肉料理", SortOrder = 1 };
        var curry = new Category { Name = "カレー", SortOrder = 2 };
        var drink = new Category { Name = "ドリンク", SortOrder = 3 };


        db.Categories.AddRange(meat, curry, drink);
        await db.SaveChangesAsync();                                // CategoryId を確定


        db.Products.AddRange(
            new Product
            {
                CategoryId = meat.Id,
```

```
                Name = "上州牛ハンバーグ",
                Price = 1480m,
                Description = "鉄板で提供するハンバーグです"
            },
            new Product
            {
                CategoryId = meat.Id,
                Name = "グリルチキン",
                Price = 1280m,
                Description = "香ばしく焼き上げたチキンです"
            },
            new Product
            {
                CategoryId = curry.Id,
                Name = "ビーフカレー",
                Price = 980m,
                Description = "じっくり煮込んだビーフカレーです"
            },
            new Product
            {
                CategoryId = drink.Id,
                Name = "アイスコーヒー",
                Price = 380m,
                Description = "食後にもおすすめです"
            });


        await db.SaveChangesAsync();                                    // 商品を保存
    }
}
```

完成コード

```
// var app = builder.Build(); の後へ追加する
if (app.Environment.IsDevelopment())
{
    using var scope = app.Services.CreateScope();
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    await SeedData.InitializeAsync(db);
}
```

コードの意味

SeedData：開発時の動作確認用データを初期登録するクラス

AnyAsync：既存データがあるか DB へ問い合わせる

CreateScope：DI から AppDbContext を取り出すためのスコープを作る

STEP 10　メニュー画面用 ViewModel を作る

対象ファイル	ViewModels/MenuViewModel.cs
作業区分	新規作成
目的	複数のデータを 1 つにまとめて View へ渡す

完成コード

```
using SelfOrderSystem.Models;
```

```
namespace SelfOrderSystem.ViewModels;


public class MenuViewModel
{
    public List<Category> Categories { get; set; } = new();      // カテゴリ一覧
    public List<Product> Products { get; set; } = new();         // 商品一覧
    public int? SelectedCategoryId { get; set; }                 // 選択中カテゴリ
}
```

STEP 11　MenuController を作る

対象ファイル	Controllers/MenuController.cs
作業区分	新規作成
目的	カテゴリと商品を DB から取得する

完成コード

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using SelfOrderSystem.Data;
using SelfOrderSystem.ViewModels;


namespace SelfOrderSystem.Controllers;


public class MenuController : Controller
{
    private readonly AppDbContext _db;                           // DB 操作用


    public MenuController(AppDbContext db)
    {
        _db = db;                                                // DI で受け取る
    }


    public async Task<IActionResult> Index(int? categoryId)
    {
        var categories = await _db.Categories
            .AsNoTracking()
            .OrderBy(c => c.SortOrder)
            .ToListAsync();


        var query = _db.Products
            .AsNoTracking()
            .Include(p => p.Category)
            .Where(p => p.IsActive);


        if (categoryId.HasValue)
        {
            query = query.Where(p => p.CategoryId == categoryId.Value);
        }


        var products = await query
            .OrderBy(p => p.Category!.SortOrder)
            .ThenBy(p => p.Name)
            .ToListAsync();


        var vm = new MenuViewModel
        {
```

```
            Categories = categories,
            Products = products,
            SelectedCategoryId = categoryId
        };

        return View(vm);                                    // ViewModel を View へ渡す
    }
}
```

コードの意味

AsNoTracking：表示専用データなので EF Core の変更追跡を行わない

Include：Product と関連する Category も同時に取得する

IQueryable：Where などの条件を追加し、ToListAsync 時に SQL として実行する

STEP 12　メニュー画面を作る

対象ファイル	Views/Menu/Index.cshtml
作業区分	新規作成
目的	カテゴリ切替と商品カードを表示する

操作手順

17. Views フォルダー内に Menu フォルダーを作成する
18. Menu フォルダーへ Index.cshtml を追加する
19. 完成コードへ置き換える

完成コード

```
@model SelfOrderSystem.ViewModels.MenuViewModel

@{
    ViewData["Title"] = "メニュー";
}

<h1 class="h3 mb-3">メニュー</h1>

<div class="d-flex gap-2 overflow-auto pb-3">
    <a class="btn @(Model.SelectedCategoryId == null ? "btn-dark" : "btn-outline-dark")"
       asp-action="Index">
        すべて
    </a>

    @foreach (var category in Model.Categories)
    {
        <a class="btn @(Model.SelectedCategoryId == category.Id ? "btn-dark" : "btn-outline-dark")"
           asp-action="Index"
           asp-route-categoryId="@category.Id">
            @category.Name
        </a>
    }
</div>
```

```
<div class="row g-3">
    @foreach (var product in Model.Products)
    {
        <div class="col-12 col-md-6 col-xl-4">
            <div class="card h-100 shadow-sm">
                <div class="card-body d-flex flex-column">
                    <h2 class="h5">@product.Name</h2>
                    <p class="text-muted flex-grow-1">@product.Description</p>
                    <p class="fs-5 fw-bold">@product.Price.ToString("N0") 円</p>

                    @if (product.IsSoldOut)
                    {
                        <span class="badge text-bg-secondary">売り切れ</span>
                    }
                </div>
            </div>
        </div>
    }
</div>
```

実行確認

- F5 で /Menu/Index が表示される
- 4 商品がカードとして表示される
- 「肉料理」「カレー」「ドリンク」を押すと商品が絞り込まれる
- SQL Server オブジェクト エクスプローラーで Categories と Products にデータが入っている

まとめ

- DB → EF Core → Controller → ViewModel → Razor View の流れで商品が表示される
- 画面表示用データは ViewModel へまとめる
- カテゴリ ID を URL へ渡すことで表示対象を絞り込める